Nineteenth Implementation Iteration: Explicit Greedy Allocation Strategy

with Batch Request Processing in the Strategy Layer

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this nineteenth implementation iteration is to make the current greedy allocation behavior explicit in the design of the prototype.

In the previous iteration, the Strategy Pattern was introduced through `IAllocationStrategy`, `GreedyAllocationStrategy`, and delegation from `AllocationService`. However, the overall greedy behavior of the system was still partly controlled by the main program, which processed loaded requests one by one in sequence.

In this iteration, the greedy algorithm is made explicit by moving batch-processing behavior into `GreedyAllocationStrategy`. As a result, the strategy now clearly represents the current allocation algorithm: process requests in loaded order, evaluate each request using the existing rule engine, create allocations immediately when a request is approved, and continue to the next request. This keeps the current behavior unchanged while making the algorithm easier to understand, replace, and extend in future iterations.

2. Changes from Iteration 18

Compared with the eighteenth iteration, the following changes were introduced:

- Extended `IAllocationStrategy` with batch-processing support through `processRequests(...)`
- Updated `GreedyAllocationStrategy` so that it explicitly processes loaded requests in order
- Added a batch-processing method to `AllocationService`
- Refactored the main program so that it uses the strategy batch path instead of directly controlling request processing
- Preserved the existing rule engine and facade for validation decisions

- Preserved request lifecycle history and existing CSV export behavior

This iteration does not introduce priority sorting, best-fit room selection, or global optimization. The implemented algorithm remains a simple greedy strategy that accepts or rejects each request immediately as it is processed.

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files through the `DataController`. The loaded requests are kept in the same order as they appear in `requests.csv`. The main program then passes the request collection to the allocation service.

The allocation service delegates the collection to the active allocation strategy. In this iteration, the selected strategy is the `GreedyAllocationStrategy`. The greedy strategy processes requests one by one in CSV order. For each request, it uses the existing `RuleEngineFacade` to check whether the request satisfies all implemented rules.

If the request passes all rules, it is approved and its allocation or allocations are created immediately. If it fails any rule, it is rejected and the rejection reason is preserved. The strategy then continues to the next request until all loaded requests have been processed. At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`
- request lifecycle history together with each exported request result

4. Modules and Their Tasks

A. Rule Evaluation Module

- Check whether each request is valid according to the implemented allocation rules
- Continue using the existing `RuleEngine` and `RuleEngineFacade`
- Return approval or failure information together with rejection reasons

B. Allocation Strategy Module

- Define how loaded requests are processed as an algorithm
- Support batch processing through `IAllocationStrategy`

- Separate the processing algorithm from the rule-checking logic

C. Greedy Allocation Module

- Process requests in the order they are loaded from CSV
- Evaluate each request using the existing rule engine facade
- Approve requests and create allocations immediately when rules pass
- Reject requests immediately when any rule fails
- Continue to the next request without reordering or optimization

D. Allocation Service Module

- Continue owning the current allocation list
- Coordinate request processing through the selected strategy
- Preserve the existing public processing methods while exposing batch processing

E. Request History and Export Module

- Preserve request lifecycle history during greedy processing
- Continue exporting accepted allocations and processed request results
- Preserve request history visibility in `request_results.csv`

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
15	Same	Same	Request priority model
16	Externalized requests	Class + Lab + Meet	DataController + CSV request loading
17	Same + history log	Class + Lab + Meet	Request lifecycle history logging + export
18	Same + strategy-based processing	Class + Lab + Meet	Allocation Strategy Pattern + Greedy strategy
19	Same + explicit greedy batch flow	Class + Lab + Meet	Greedy batch processing moved into strategy

6. Updated Class Diagram

Figure 1 shows the updated class diagram including batch-processing support in `IAllocationStrategy`, the explicit greedy strategy behavior, and the updated service interaction.

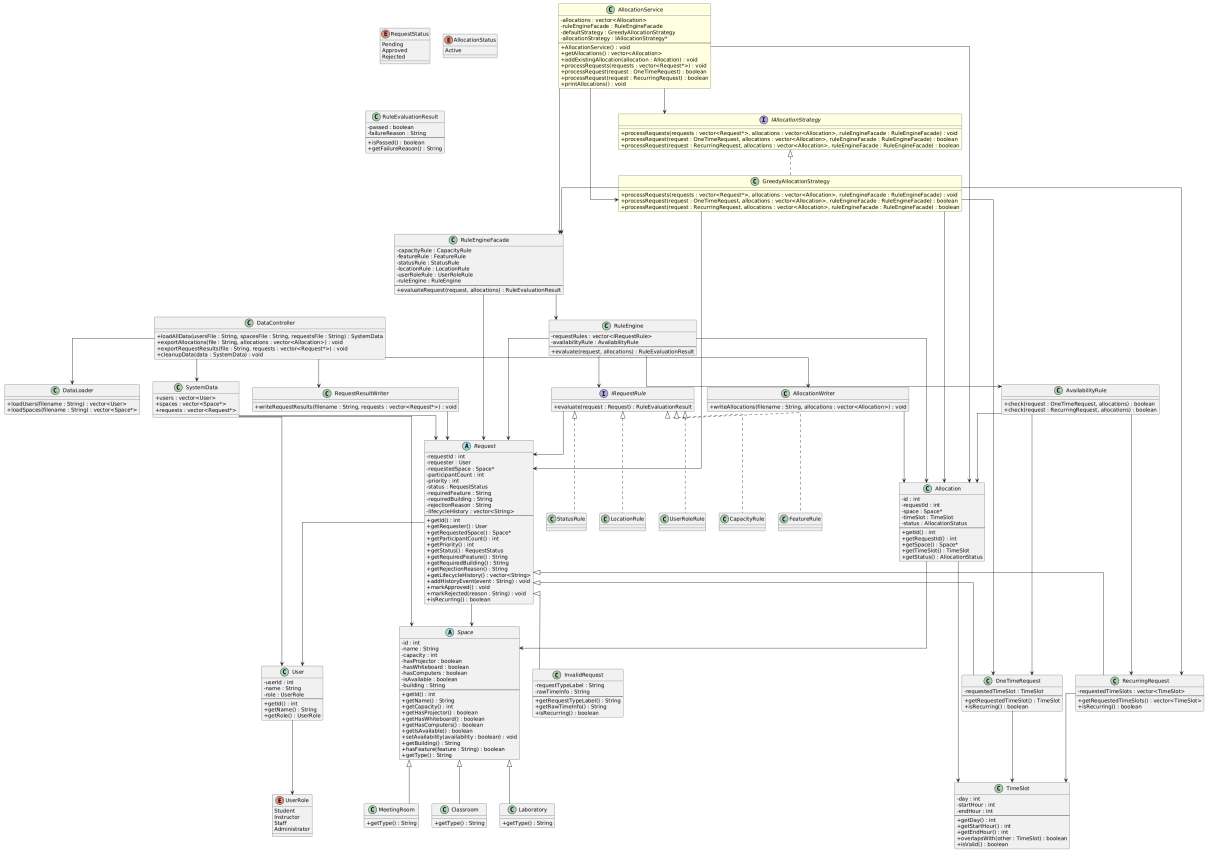

Figure 1: Updated Class Diagram with Explicit Greedy Allocation Strategy

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The main program now delegates the loaded request collection to the allocation service, which then passes control to the greedy strategy for ordered processing.

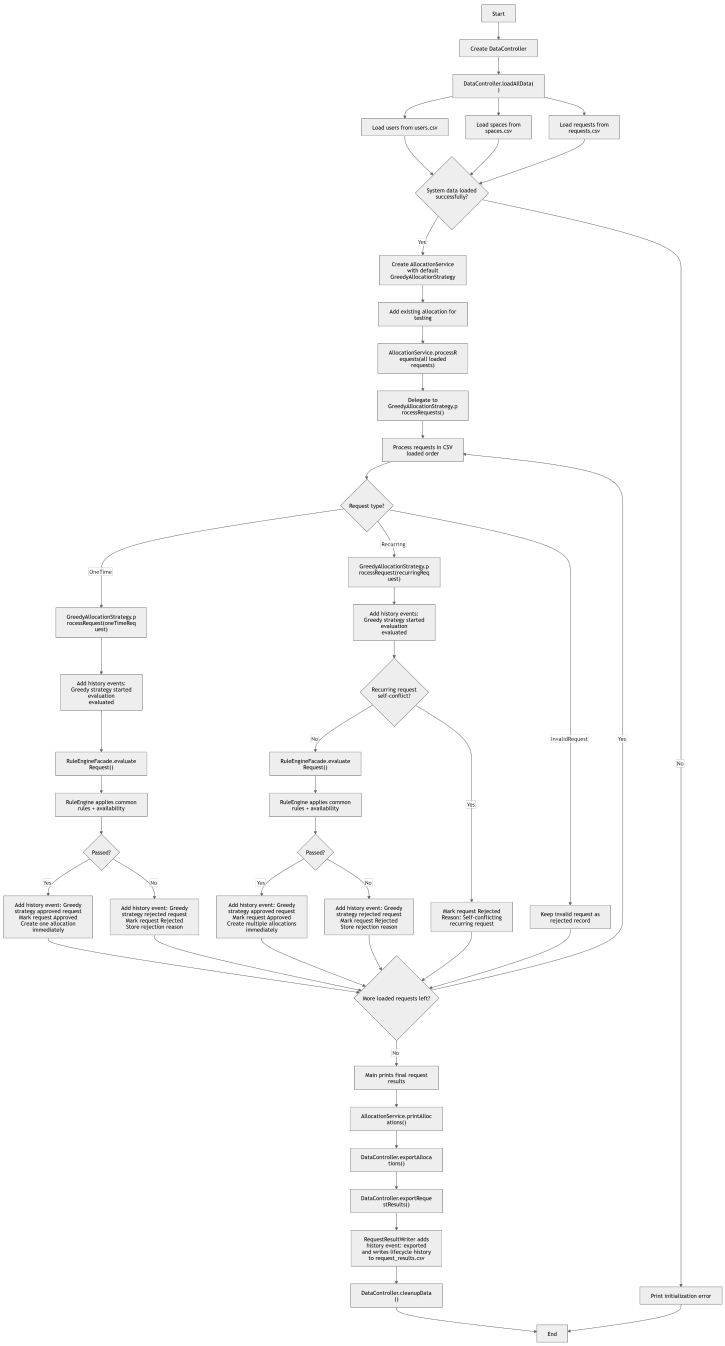

Figure 2: Workflow with Explicit Greedy Batch Processing

8. Prototype Implementation

The C++ prototype was extended by making the greedy allocation algorithm explicit inside the strategy layer. The `IAllocationStrategy` interface now supports batch processing of loaded requests through `processRequests(...)`. The `GreedyAllocationStrategy` implements this method by processing requests in the order they were loaded from the CSV input.

This design makes the difference between rules and strategies clearer. The rules still decide whether a request is valid, while the strategy decides how requests are processed as an algorithm. In this iteration, the selected algorithm is simple greedy processing: evaluate the current request, approve and allocate immediately if it passes, reject immediately if it fails, and continue to the next request.

The `AllocationService` was updated to expose batch processing while continuing to own the current allocation list. The main program was simplified so that it now uses the strategy-based batch path instead of manually controlling request processing itself.

This iteration does not change the rule set and does not add optimization behavior such as priority sorting or best-fit selection. Instead, it provides a clearer architectural foundation for future strategies such as a priority-based strategy, a best-fit strategy, or a priority-aware best-fit strategy.

9. Summary

This nineteenth implementation iteration makes the greedy allocation algorithm explicit in the architecture of the prototype. Request processing in loaded CSV order is now clearly represented inside `GreedyAllocationStrategy` rather than being only implied by the main program flow.

This iteration preserves the current functional behavior of the system while improving the separation between validation rules and processing strategies. As a result, the prototype is now better prepared for future iterations that introduce more advanced allocation algorithms without redesigning the rest of the system.